

Institut National des Postes et
Télécommunications

INPT

Rapport de Travaux Pratiques

Communication Peer-to-Peer en Temps Réel

Réalisé par :

Reda ELHADFI

Alae LAITA

Encadré par :

Professeur Asma EL HAMZAOUI

INE3

Année Universitaire 2025-2026

Février 2026

Table des matières

1	Introduction	3
1.1	Objectifs pédagogiques	3
2	TP1 : Les Servlets HTTP	4
2.1	Présentation générale	4
2.2	Architecture et structure	4
2.3	Les servlets développés	4
2.3.1	GreetingServlet - Version de base	4
2.3.2	LotteryServletJDBC - Intégration base de données	4
2.3.3	SecureGreetingServlet - Authentification	5
2.3.4	BlacklistFilter - Filtrage des requêtes	5
2.4	Extensions avec JSP	5
2.5	Gestion des erreurs	5
2.6	Difficultés rencontrées	5
3	TP2 : Plateforme Node.js	7
3.1	Introduction à Node.js	7
3.2	Les 6 versions progressives	7
3.2.1	Version 1 : Hello World	7
3.2.2	Version 2 : Modularisation	7
3.2.3	Version 3 : Routage	7
3.2.4	Version 4 : Request Handlers	8
3.2.5	Version 5 : Retour de contenu	8
3.2.6	Version 6 : Asynchronisme	8
3.3	Application complète avec upload	8
3.4	Application Socket.io - Chat temps réel	8
3.5	Application MEAN Stack	9
3.5.1	Backend avec Express et MongoDB	9
3.5.2	Frontend avec Angular 17	9
3.5.3	Intégration Frontend-Backend	10
3.6	Docker et MongoDB	10
3.7	Ce qu'on a appris	10
4	TP3 : WebRTC	11
4.1	Introduction à WebRTC	11
4.2	Étape 1 : getUserMedia	11
4.3	Étape 2 : RTCDataChannel	11
4.4	Étape 3 : Serveur de signalisation	12
4.5	Étape 4 : Application complète	12

4.5.1	Interface utilisateur	13
4.5.2	Gestion des tracks	13
4.6	Difficultés techniques	13
4.7	Points techniques importants	13
4.7.1	SDP (Session Description Protocol)	13
4.7.2	ICE (Interactive Connectivity Establishment)	14
4.7.3	NAT Traversal	14
4.8	Résultats et tests	14
4.9	Ce qu'on a appris	16
5	Conclusion	17
5.1	Synthèse des apprentissages	17
5.2	Compétences acquises	17
5.3	Difficultés et solutions	17
5.4	Perspectives d'amélioration	18
5.5	Remerciements	18

Chapitre 1

Introduction

Dans le cadre de notre formation en troisième année à l'INPT, nous avons réalisé une série de travaux pratiques axés sur la communication peer-to-peer et les technologies web modernes. Ces TP nous ont permis d'explorer trois domaines complémentaires : les servlets Java, Node.js, et WebRTC.

Ce rapport présente le travail effectué durant ces trois TP, en détaillant les concepts appris, les difficultés rencontrées, et les solutions que nous avons mises en place. Notre objectif était de maîtriser les technologies du web en temps réel, depuis les bases de la programmation serveur jusqu'aux communications vidéo peer-to-peer.

1.1 Objectifs pédagogiques

Les objectifs principaux de ces travaux pratiques étaient de :

- Comprendre l'architecture client-serveur et les servlets Java
- Maîtriser Node.js et ses frameworks (Express, Socket.io)
- Développer une application full-stack avec la stack MEAN
- Implémenter des communications en temps réel
- Explorer WebRTC pour les communications peer-to-peer

Chapitre 2

TP1 : Les Servlets HTTP

2.1 Présentation générale

Le premier TP nous a permis de découvrir la plateforme Java EE et plus particulièrement les servlets HTTP. On a développé une application de loterie virtuelle qui illustre plusieurs concepts fondamentaux du développement web côté serveur.

L'idée était simple mais efficace : l'utilisateur saisit son nom via un formulaire HTML, et le serveur lui attribue un gain aléatoire tout en affichant un message personnalisé. Cette simplicité apparente cache en réalité plusieurs aspects techniques qu'on a dû maîtriser.

2.2 Architecture et structure

Notre application suit la structure standard d'une application JEE. On a organisé le projet avec un dossier `src/` pour le code Java, un dossier `WebContent/` pour les pages web, et surtout le fameux `WEB-INF/` qui contient la configuration et les classes compilées.

Au début, on avait du mal à comprendre pourquoi il fallait respecter cette structure si rigide, mais on s'est vite rendu compte que c'est ce qui permet à Tomcat de déployer correctement l'application.

2.3 Les servlets développés

2.3.1 GreetingServlet - Version de base

Notre premier servlet était assez basique. Il récupère le nom de l'utilisateur depuis le formulaire, génère un nombre aléatoire pour le gain, et construit une page HTML en réponse. On a utilisé les méthodes `doGet()` et `doPost()` pour gérer les requêtes.

Ce qui était intéressant, c'est qu'on a dû générer tout le HTML directement dans le code Java avec des `PrintWriter`. C'est pas très élégant mais ça nous a fait comprendre comment fonctionne la génération dynamique de contenu.

2.3.2 LotteryServletJDBC - Intégration base de données

Ensuite, on a ajouté la persistance des données avec MySQL. Là, ça devenait plus complexe parce qu'il fallait gérer les connexions à la base de données, exécuter des requêtes SQL, et surtout bien fermer les ressources.

On a galéré un moment avec les `SQLException` et les problèmes de connexion. Il fallait bien penser à fermer les `Connection`, `PreparedStatement` et `ResultSet` dans des blocs `finally`, sinon on se retrouvait avec des fuites de ressources.

Le servlet affiche maintenant l'historique des gains précédents, ce qui rend l'application plus intéressante.

2.3.3 SecureGreetingServlet - Authentification

Pour la version sécurisée, on a implémenté un mécanisme d'authentification BASIC HTTP. On a configuré un realm dans Tomcat avec le fichier `tomcat-users.xml`, et on a ajouté des contraintes de sécurité dans le `web.xml`.

C'était notre première approche de la sécurité web, et on a compris l'importance de protéger certaines ressources. L'utilisateur doit maintenant s'authentifier avant d'accéder au servlet.

2.3.4 BlacklistFilter - Filtrage des requêtes

Le filtre qu'on a créé permet de bloquer certains noms avant même que la requête n'arrive au servlet. On a défini une liste noire dans le `web.xml`, et le filtre intercepte les requêtes pour vérifier si le nom est interdit.

C'était intéressant de voir comment les filtres s'insèrent dans la chaîne de traitement des requêtes. Ça nous a aussi fait réfléchir à la validation des entrées utilisateur.

2.4 Extensions avec JSP

On a aussi développé des versions JSP de notre application. Contrairement aux servlets où on générait tout le HTML en Java, avec JSP on peut mélanger du HTML et du code Java avec des balises spéciales.

Honnêtement, c'est beaucoup plus pratique pour la présentation. Le code est plus lisible et la séparation entre la logique et la présentation est meilleure. On a fait deux versions : une simple et une avancée avec plus de styles CSS.

2.5 Gestion des erreurs

On a créé des pages d'erreur personnalisées pour les codes HTTP courants (404, 403, 500). Au lieu d'afficher les pages d'erreur standard de Tomcat qui sont plutôt moches, on a nos propres pages avec un design cohérent.

Ça améliore vraiment l'expérience utilisateur quand quelque chose se passe mal.

2.6 Difficultés rencontrées

La principale difficulté était la configuration de Tomcat. Entre le `web.xml`, le `tomcat-users.xml`, et les chemins de déploiement, il y avait beaucoup de choses à configurer correctement.

On a aussi eu quelques soucis avec MySQL au début, notamment pour configurer le driver JDBC dans le classpath. Il fallait copier le fichier JAR dans `WEB-INF/lib/`, ce qu'on avait oublié la première fois.

Application de Loterie Virtuelle

TP1 - Les Servlets HTTP | Communication P2P en Temps Réel sur le Web

À propos de ce projet

Cette application démontre l'utilisation des servlets HTTP, JSP, JDBC, filtres, et mécanismes de sécurité dans une application JEE. Chaque version illustre des concepts différents du développement web Java Enterprise Edition.



Version de Base HTML

Application simple avec formulaire HTML et servlet de base.

- ✓ Formulaire HTML statique
- ✓ Servlet HttpServlet de base
- ✓ Méthodes GET et POST
- ✓ Génération dynamique HTML

Lancer l'application



Version JSP Simple JSP

Interface améliorée avec JavaServer Pages et styles modernes.

- ✓ JSP avec scriptlets
- ✓ Design moderne avec CSS
- ✓ Résultat sur la même page
- ✓ Informations de session

Lancer l'application



Version JSP Avancée Avancé

Version JSP avec affichage détaillé des informations de session et requête.

- ✓ Tableau détaillé de session
- ✓ Informations HTTP complètes
- ✓ IP client et User-Agent
- ✓ Date et heure

Lancer l'application



Version avec JDBC MySQL

Intégration avec base de données MySQL.

Lancer l'application



Version Sécurisée Auth

Application avec authentification et contrôle d'accès basé sur les rôles.

Lancer l'application



Version avec Filtre Filter

Démonstration des filtres avec liste noire d'utilisateurs.

Lancer l'application

- ✓ Connexion MySQL via JDBC
- ✓ Sauvegarde des résultats
- ✓ Historique des participations
- ✓ Gestion des ressources

Lancer l'application

- ✓ Authentification HTTP BASIC
- ✓ Rôle "tomcat" requis
- ✓ Contraintes de sécurité
- ✓ Realm Tomcat

Lancer l'application

- ✓ Filtre Java personnalisé
- ✓ Blacklist configurable
- ✓ Interception de requêtes
- ✓ Page de refus d'accès

Lancer l'application



Méthodes HTTP Demo

Démonstration de toutes les méthodes HTTP implémentées.

- ✓ GET, POST, PUT, DELETE
- ✓ HEAD, OPTIONS, TRACE
- ✓ Overridé de service()
- ✓ Tests interactifs

Lancer la démo

Configuration Requisite

Pour JDBC: MySQL Server + base de données lottery_db
Pour Sécurité: Configuration tomcat-users.xml
Pour Filtre: Aucune configuration supplémentaire requise

TP1 - Les Servlets HTTP
Communication P2P en temps réel sur le web | INE3 - INPT

[Documentation](#) | [Guide Rapide](#) | [Tests](#)

Chapitre 3

TP2 : Plateforme Node.js

3.1 Introduction à Node.js

Le deuxième TP représentait un changement complet de paradigme pour nous. On passait de Java et ses serveurs lourds à Node.js, qui est beaucoup plus léger et basé sur JavaScript. C'était vraiment différent de ce qu'on connaissait.

Node.js utilise un modèle asynchrone non-bloquant, ce qui change complètement la façon de programmer. Au lieu d'avoir des threads qui attendent, tout passe par des callbacks et des événements.

3.2 Les 6 versions progressives

On a commencé très simplement avec 6 versions qui nous ont fait progresser étape par étape.

3.2.1 Version 1 : Hello World

Notre premier serveur Node.js était minimaliste : juste quelques lignes de code pour créer un serveur HTTP qui répond "Hello World". Mais même là, on voyait déjà la puissance de Node.js. Le serveur démarre en une seconde, et le code est super simple comparé à une servlet Java.

3.2.2 Version 2 : Modularisation

On a appris à organiser le code en modules avec `require()` et `exports`. On a séparé la création du serveur, le routage, et la gestion des requêtes dans différents fichiers. C'est important pour avoir un code maintenable.

3.2.3 Version 3 : Routage

Ici, on a implémenté un système de routage basique. Selon l'URL demandée, on renvoie du contenu différent. On gérait les routes comme `"/start"` et `"/upload"` manuellement en parcourant l'URL.

3.2.4 Version 4 : Request Handlers

On a créé des fonctions dédiées pour gérer chaque route. C'était notre première approche d'une architecture MVC-like. Chaque handler s'occupe d'une fonctionnalité spécifique.

3.2.5 Version 5 : Retour de contenu

Cette version introduisait le retour de contenu dynamique. On ne se contentait plus de réponses statiques, on commençait à générer du HTML en fonction des paramètres de la requête.

3.2.6 Version 6 : Asynchronisme

La version 6 était la plus complexe. On a vraiment plongé dans l'asynchronisme avec des callbacks. On a compris pourquoi Node.js est si performant : pendant qu'une opération attend (lecture fichier, base de données), le serveur peut traiter d'autres requêtes.

Au début, on s'emmêlait avec les callbacks imbriqués (le fameux "callback hell"), mais on a fini par comprendre le pattern.

3.3 Application complète avec upload

L'application complète intégrait tout ce qu'on avait appris. On pouvait uploader des fichiers grâce à la bibliothèque `formidable`. L'upload de fichiers en Node.js est assez différent de ce qu'on faisait en Java, mais une fois qu'on a compris le principe des streams, c'est devenu clair.

3.4 Application Socket.io - Chat temps réel

C'était probablement la partie la plus fun du TP2. On a créé un chat où plusieurs utilisateurs peuvent communiquer en temps réel. Socket.io gère tout le protocole WebSocket pour nous.

Ce qui est impressionnant, c'est la simplicité du code. Côté serveur, on écoute les événements et on les broadcast aux autres clients. Côté client, on émet des événements et on écoute les réponses. Et boom, on a un chat fonctionnel !

On a géré plusieurs fonctionnalités :

- Connexion/déconnexion des utilisateurs
- Messages de chat publics
- Notification quand quelqu'un rejoint ou quitte
- Liste des utilisateurs connectés
- Messages privés entre utilisateurs

L'interface est simple mais efficace, avec un peu de CSS pour que ça ait l'air professionnel.

3.5 Application MEAN Stack

La partie MEAN Stack était la plus ambitieuse du TP. MEAN signifie MongoDB, Express, Angular, Node.js. C'est une stack complète pour développer des applications web modernes.

3.5.1 Backend avec Express et MongoDB

Côté backend, on a utilisé Express qui est un framework web pour Node.js. C'est beaucoup plus pratique que de tout coder à la main comme dans les 6 premières versions.

On a structuré notre API REST avec les endpoints suivants :

- **POST /api/register** : inscription d'un nouvel utilisateur
- **POST /api/login** : connexion et récupération du token JWT
- **GET /api/profile** : récupération du profil utilisateur
- **PUT /api/profile** : mise à jour du profil
- **GET /api/users** : liste de tous les utilisateurs
- **DELETE /api/users/:id** : suppression d'un utilisateur
- **POST /api/upload** : upload de fichier
- **POST /api/upload-avatar** : upload d'avatar

Pour la base de données, on utilise MongoDB avec Mongoose. Au début, on trouvait bizarre de ne pas avoir de schéma SQL strict, mais on s'est habitués à la flexibilité de MongoDB. On a défini des schémas Mongoose pour structurer quand même nos données.

La partie authentification avec JWT était nouvelle pour nous. Le principe est simple : après la connexion, on génère un token signé qui contient l'ID de l'utilisateur. Le client envoie ce token dans les headers de chaque requête, et on peut vérifier qui fait la requête.

On a sécurisé les mots de passe avec bcrypt. On ne stocke jamais les mots de passe en clair, on les hash avec un salt aléatoire.

3.5.2 Frontend avec Angular 17

Le frontend en Angular était un gros morceau. Angular 17 est vraiment moderne et puissant, mais il y a beaucoup à apprendre.

On a créé plusieurs composants :

- **Login** : formulaire de connexion avec validation
- **Register** : inscription avec vérification du mot de passe
- **Dashboard** : vue d'ensemble avec statistiques
- **Profile** : gestion du profil et upload de fichiers
- **Navbar** : navigation adaptée selon l'état de connexion

On a utilisé les Reactive Forms pour les formulaires. C'est plus verbeux que les template-driven forms, mais beaucoup plus puissant pour la validation complexe. Par exemple, pour le register, on vérifie que les deux mots de passe correspondent avec un validator custom.

Les services Angular (AuthService, UserService, FileService) encapsulent toute la logique de communication avec le backend. On utilise les Observables de RxJS, ce qui est super pratique pour gérer les requêtes asynchrones.

Le AuthGuard protège les routes privées. Si quelqu'un essaie d'accéder au dashboard sans être connecté, il est automatiquement redirigé vers le login.

L'interceptor HTTP est génial : il ajoute automatiquement le token JWT dans le header de toutes les requêtes. Si on reçoit une erreur 401 (non autorisé), il déconnecte automatiquement l'utilisateur.

3.5.3 Intégration Frontend-Backend

L'intégration entre Angular et Express s'est plutôt bien passée. On a configuré les CORS sur le backend pour accepter les requêtes depuis localhost :4200.

Au début, on avait quelques erreurs de type dans TypeScript parce que les interfaces côté Angular ne correspondaient pas exactement à ce que renvoyait le backend. On a dû bien définir nos interfaces TypeScript.

3.6 Docker et MongoDB

On a conteneurisé MongoDB avec Docker, ce qui facilite grandement le déploiement. Plus besoin d'installer MongoDB localement, on lance juste le container et ça fonctionne.

Le fichier `docker-compose.yml` définit tout : l'image MongoDB, les ports, les variables d'environnement, et les volumes pour persister les données.

3.7 Ce qu'on a appris

Ce TP nous a vraiment fait comprendre l'écosystème JavaScript moderne. De Node.js à Angular en passant par MongoDB, on a touché à tout. La programmation asynchrone était un défi, mais maintenant on comprend mieux les promesses, les callbacks, et les Observables.

Angular nous a montré comment structurer une grosse application frontend. Le système de modules, les composants réutilisables, les services injectables, c'est une vraie architecture professionnelle.

Chapitre 4

TP3 : WebRTC

4.1 Introduction à WebRTC

Le troisième TP était consacré à WebRTC (Web Real-Time Communication), une technologie qui permet la communication peer-to-peer directement entre navigateurs. Contrairement à Socket.io où tout passe par le serveur, WebRTC établit une connexion directe entre les clients.

C'était le TP le plus impressionnant visuellement, parce qu'à la fin on a une vraie application de visioconférence !

4.2 Étape 1 : getUserMedia

La première étape consistait à accéder à la webcam et au microphone de l'utilisateur. L'API `getUserMedia` permet de capturer des flux audio et vidéo.

Le code est assez simple :

```
1 navigator.mediaDevices.getUserMedia({ video: true, audio: true })
2   .then(function(stream) {
3     // On a le flux, on peut l'afficher
4     videoElement.srcObject = stream;
5   })
6   .catch(function(error) {
7     console.error('Erreur:', error);
8   });
```

Au début, on avait oublié que le navigateur demande la permission à l'utilisateur. Il faut cliquer sur "Autoriser" dans la popup, sinon ça ne marche pas. Aussi, `getUserMedia` ne fonctionne qu'en HTTPS ou sur localhost pour des raisons de sécurité.

4.3 Étape 2 : RTCDataChannel

Cette étape nous faisait découvrir `RTCPeerConnection`, l'objet central de WebRTC. On a créé un data channel pour échanger des messages texte entre deux peers.

La partie compliquée, c'est le processus de signalisation. Les deux peers doivent échanger des informations avant d'établir la connexion :

1. Le peer A crée une offre (SDP offer)

2. Le peer B reçoit l'offre et crée une réponse (SDP answer)
3. Les deux peers échangent des ICE candidates pour trouver le meilleur chemin réseau

Dans cette étape, on faisait la signalisation manuellement en copiant-collant les SDP et ICE candidates d'un peer à l'autre. C'est fastidieux mais ça nous a fait comprendre ce qui se passe sous le capot.

4.4 Étape 3 : Serveur de signalisation

Pour automatiser la signalisation, on a créé un serveur avec Socket.io. Ce serveur joue le rôle d'intermédiaire pour échanger les informations de connexion entre les peers.

Le flow devient :

1. Peer A se connecte au serveur de signalisation
2. Peer B se connecte au serveur
3. Peer A crée une offre et l'envoie au serveur
4. Le serveur transmet l'offre à Peer B
5. Peer B crée une réponse et l'envoie au serveur
6. Le serveur transmet la réponse à Peer A
7. Les ICE candidates sont échangés de la même manière
8. Une fois la connexion établie, les peers communiquent directement

Le serveur est assez simple, il fait juste du relais de messages :

```
1 socket.on('offer', function(data) {  
2     socket.broadcast.emit('offer', data);  
3 });  
4  
5 socket.on('answer', function(data) {  
6     socket.broadcast.emit('answer', data);  
7 });
```

Important : le serveur ne voit pas le contenu des communications après l'établissement de la connexion. Les données passent directement d'un peer à l'autre.

4.5 Étape 4 : Application complète

L'application finale combine tout : vidéo, audio, et chat texte. On a créé une interface avec deux sections principales :

- La zone vidéo avec notre propre flux et celui du peer distant
- Le chat texte à côté pour communiquer par messages

On a géré plusieurs événements :

- Connexion/déconnexion des peers
- Démarrage/arrêt de la vidéo et du micro
- Envoi et réception de messages de chat
- Gestion des erreurs de connexion

4.5.1 Interface utilisateur

On a fait une interface assez propre avec CSS. Les vidéos s'affichent en grille, avec la vidéo locale plus petite dans un coin. Le chat est sur le côté avec un champ de saisie en bas.

On a ajouté des boutons pour contrôler la vidéo et le micro :

- Toggle vidéo on/off
- Toggle micro on/off
- Raccrocher

4.5.2 Gestion des tracks

Un point délicat était la gestion des tracks audio et vidéo. Quand on coupe le micro, il faut désactiver le track audio sans fermer toute la connexion. Pareil pour la vidéo.

```
1 function toggleVideo() {  
2     const videoTrack = localStream.getVideoTracks()[0];  
3     videoTrack.enabled = !videoTrack.enabled;  
4     // Mettre à jour l'icône du bouton  
5 }
```

4.6 Difficultés techniques

WebRTC a ses complexités. Le principal problème qu'on a rencontré, c'est la traversée des NAT. En local ça marchait bien, mais quand on essayait depuis des réseaux différents, la connexion ne s'établissait pas toujours.

C'est là qu'interviennent les serveurs STUN et TURN :

- **STUN** aide à découvrir l'adresse IP publique
- **TURN** relaie les données quand la connexion directe est impossible

On a configuré un serveur STUN public dans notre code :

```
1 const configuration = {  
2     iceServers: [  
3         { urls: 'stun:stun.l.google.com:19302' }  
4     ]  
5 };
```

Un autre point compliqué était la gestion des états de connexion. WebRTC a plusieurs états (new, connecting, connected, disconnected, failed), et il faut les gérer proprement pour informer l'utilisateur.

4.7 Points techniques importants

4.7.1 SDP (Session Description Protocol)

Le SDP contient toutes les informations sur la session : codecs supportés, résolution vidéo, échantillonnage audio, etc. C'est un format texte assez verbeux qu'on n'a pas eu à manipuler directement, mais c'est intéressant de comprendre ce qu'il contient.

4.7.2 ICE (Interactive Connectivity Establishment)

ICE est le protocole qui trouve le meilleur chemin réseau entre les peers. Il teste différentes routes possibles et choisit la plus rapide. Les ICE candidates sont les différentes adresses IP/port où le peer peut être contacté.

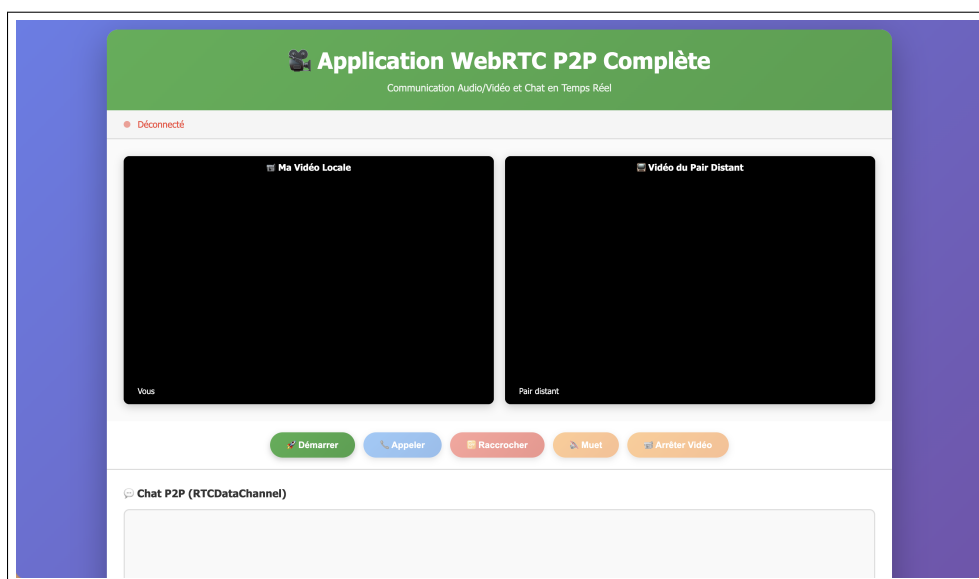
4.7.3 NAT Traversal

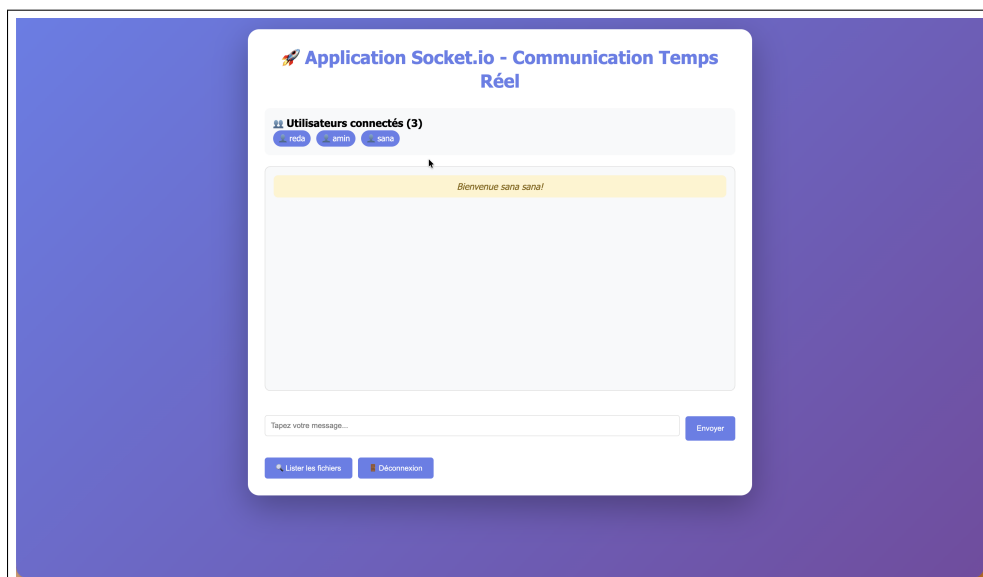
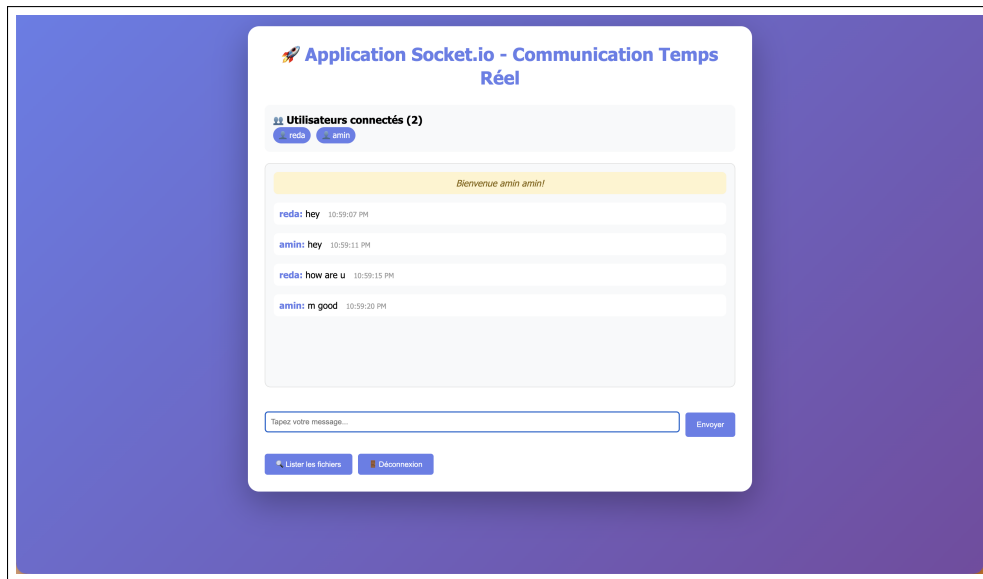
La traversée des NAT est un gros challenge. La plupart des utilisateurs sont derrière un routeur qui fait du NAT. WebRTC essaie plusieurs techniques pour établir la connexion : direct, STUN-assisted, ou via TURN en dernier recours.

4.8 Résultats et tests

On a testé l'application avec deux navigateurs sur le même ordinateur d'abord, puis sur deux ordinateurs différents du même réseau. La qualité vidéo était bonne, avec un petit délai (latence) d'environ 200-300ms.

Le chat texte via le data channel était quasi instantané. On pouvait voir l'autre personne taper en temps réel.





```
Socket.io server running on http://localhost:3000
New user registered: reda
A user connected: -As6zFCak-86Yv6vAAAB
Message from reda : reda
Message from reda : hello
New user registered: amin
A user connected: 7ZqrKD6mWp0kAKqtAAAD
Message from amin : hey this is test message
User disconnected: amin
A user connected: jHZ-cBiVLXyWcgv6AAAF
User disconnected: reda
A user connected: fs_g1xvjIhub8pFXAAAH
Message from amin : hello how are u ?
Message from amin : hey m good thanks , wbu ?
User disconnected: amin
A user connected: XKREBv0eEAMgXlV_AA AJ
User disconnected: amin
A user connected: _NIJ-tfYyBV7gKnHAAAL
Message from reda : hey
Message from amin : hey
Message from reda : how are u
Message from amin : m good
New user registered: sana
A user connected: DK8VU4dYDkDEKhXtAAAN
rver
```

4.9 Ce qu'on a appris

Ce TP nous a ouvert les yeux sur la complexité des communications en temps réel. Entre les protocoles de signalisation, la négociation ICE, les codecs audio/vidéo, et la gestion des erreurs, il y a beaucoup de choses à maîtriser.

Mais c'est aussi très gratifiant de voir une vraie application de visioconférence qu'on a codée nous-mêmes fonctionner !

Chapitre 5

Conclusion

5.1 Synthèse des apprentissages

Ces trois TPs nous ont fait parcourir un large spectre de technologies web. On est partis des servlets Java classiques, on a exploré Node.js et son écosystème, et on a fini avec WebRTC et les communications peer-to-peer.

Chaque TP apportait son lot de challenges :

- TP1 : comprendre les bases du développement web côté serveur
- TP2 : maîtriser JavaScript et l’asynchronisme
- TP3 : implémenter des communications temps réel complexes

5.2 Compétences acquises

Au niveau technique, on a acquis des compétences variées :

- Architecture client-serveur et API REST
- Gestion des bases de données (MySQL et MongoDB)
- Authentification et sécurité (JWT, bcrypt)
- Frameworks modernes (Express, Angular)
- Communication temps réel (Socket.io, WebRTC)
- Conteneurisation (Docker)

Mais au-delà des aspects techniques, on a surtout appris à apprendre. Chaque technologie a sa documentation, sa communauté, ses best practices. Savoir se débrouiller face à une erreur, chercher dans la doc, tester des solutions, c’est aussi important que connaître la syntaxe.

5.3 Difficultés et solutions

Les principales difficultés qu’on a rencontrées :

Configuration des environnements : Entre Tomcat, Node.js, MongoDB, et tous les outils, il y avait beaucoup de configuration. On a appris à bien lire les messages d’erreur et à chercher dans les logs.

Debugging asynchrone : Avec Node.js et les callbacks, le debugging était parfois compliqué. On a utilisé beaucoup de `console.log()` au début, puis on a découvert les outils de debug de Chrome et VS Code.

CORS et sécurité : Les problèmes CORS entre Angular et Express nous ont fait comprendre l'importance de la sécurité web. On a appris à configurer correctement les headers.

WebRTC et NAT : La connexion WebRTC entre différents réseaux nécessitait de comprendre les serveurs STUN/TURN.

5.4 Perspectives d'amélioration

Si on devait continuer ces projets, on pourrait :

- Ajouter des tests unitaires et d'intégration
- Implémenter un système de rooms pour le chat et la visio
- Améliorer l'UI/UX avec des frameworks comme Tailwind CSS
- Déployer les applications sur un serveur cloud
- Ajouter plus de fonctionnalités (partage d'écran, enregistrement, etc.)
- Optimiser les performances et gérer la montée en charge

5.5 Remerciements

On tient à remercier notre professeure Asma El Hamzaoui pour son encadrement durant ces TPs. Les sujets étaient bien construits et progressifs, ce qui nous a permis de bien assimiler chaque concept avant de passer au suivant.

Ces travaux pratiques nous ont donné une base solide pour nos futurs projets et notre future carrière dans le développement web.